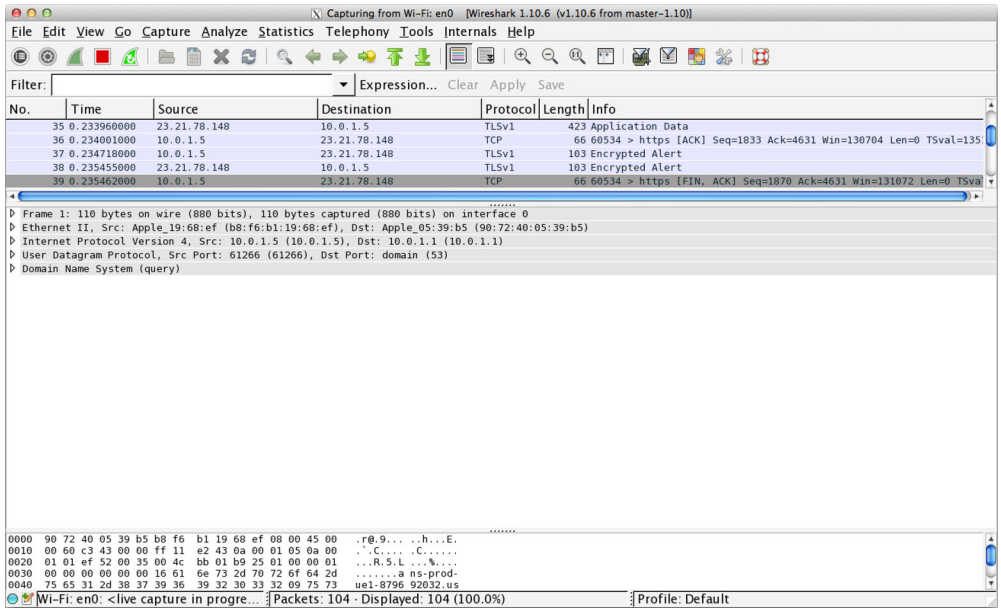


Figure 7.8 Wireshark capturing network traffic

Reloading the page will trigger an HTTP request to your server. Once the page has reloaded, you can stop the capture in Wireshark by clicking Stop.

The CORS request is logged somewhere in the network capture log, but finding it can be like spotting a needle in a haystack. Luckily Wireshark provides a rich filtering language to help narrow down the entries in the log. A simple way to find the CORS request is to filter the network log by HTTP requests. This is as simple as typing `http` in the filter box. Figure 7.9 shows the results of filtering by HTTP requests. You can clearly see both the request and response for the client.html page and the /api/posts pages.

Notice that unlike in Chrome's developer tools, you can see the response to the /api/posts request. Clicking each log item will show more details about the request, including the headers. If you click the response to the /api/posts requests, you can see that although the request has an Origin header, the response doesn't have an Access-Control-Allow-Origin header, and therefore the request fails. Figure 7.10 shows the discrepancy in the headers.

Wireshark is useful for digging into HTTP requests when the browser doesn't provide the information you need. This is often the case when making simple `GET` or `POST` requests that don't have a preflight request.

7.3.2 Using Fiddler

Fiddler is another packet analyzer. It is easier to use than Wireshark, but it's available only on Windows. In addition to network analysis, Fiddler supports a lot of other features, such as web debugging, web session manipulation, performance testing, and